Appendices

Appendix 1: *Data of used material. Starred* values have been estimated to be the same as last year. Double-starred** values have been estimated using the linear FORECAST function of Libreoffice Calc.*

Year	Gray seal population of the Baltic sea	Sea fishing	River fishing of Tornionjoki	Measured salmon migration data of Tornionjoki
2009	15313	176797	13400	31782
2010	19627	147706	8600	17224
2011	23941	143267	10100	23076
2012	28255	126579	20000	59606
2013	29496	101755	15200	53605
2014	32244	99464	24200	101094
2015	30285	92747	23200	58360
2016	30070	88449	29200	98311
2017	30348	80657	18000	40952
2018	34235	93457	21100	47028
2019	38121	91234	20900	65520
2020	40075	76095	23300	69149
2021	41936	86796	30900	93121
2022	36761	48438	19800	52030
2023	45798	36554	10900	20245
2024	48711	33048	10800	24627
2025	48087**	33048*	10800*	45043

Appendix 2: *Table of the variation of seals and fishing. During the simulation the constant parameter was 1.0. Seal amount has different values for each year because of the FORECAST function in Libreoffice Calc. These values were multiplied by the multiplier.*

Multiplier	Seal amount	Sea fishing	River fishing
0.0	0	0	0
0.5	-	16524	5400
1.0	-	33048	10800
1.5	-	49572	16200
2.0	-	66096	21600

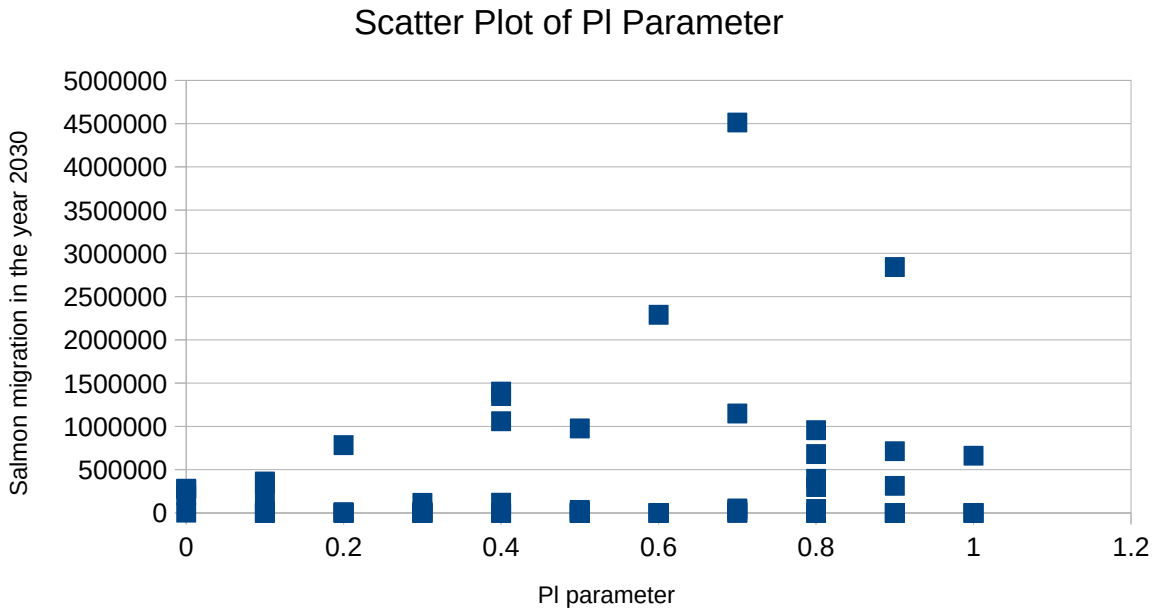
Appendix 3: *The ranges for which the parameter search was done.*

Parameter	Minimum value	Maximum value	Step
Pl	0	0.9	0.1
Pj	0	0.9	0.1
Alpha ()	0	4.0	0.1
Gamma ()	0	500000	10000
Fecundity	0	4000	100

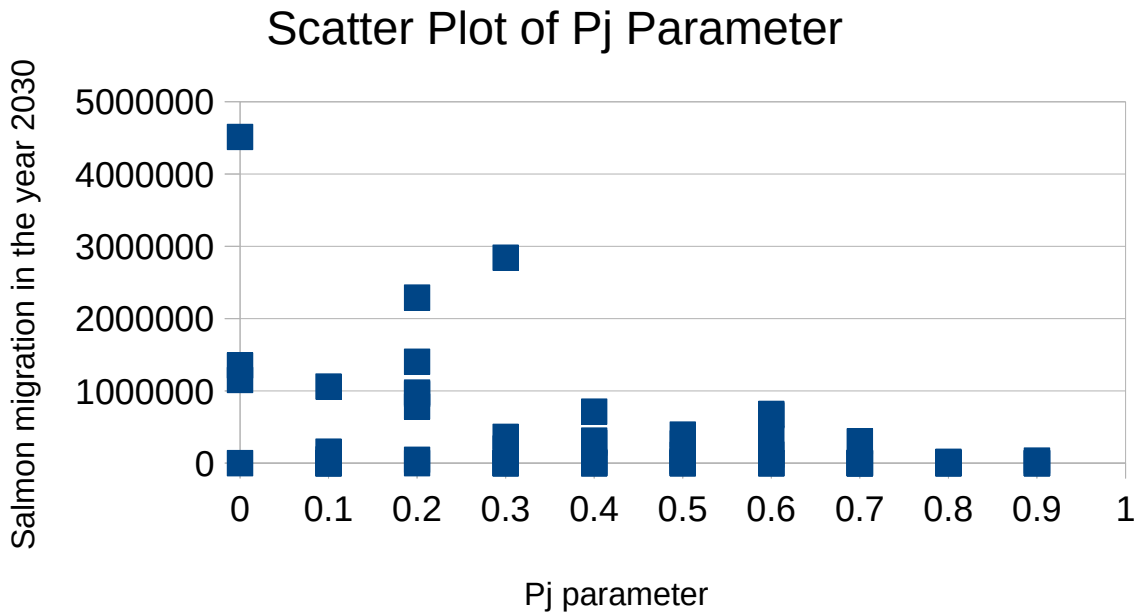
Appendix 4: *Values of nearly orthogonal latin hypercube.*

Pl	Pj	Alpha	Gamma	Fecundity
0.7	0	0.4	385625	589
1	0.6	0.1	404375	595
0.9	0.3	1	363750	594
0.6	0.8	0.7	410625	587
0.9	0.4	0.2	323125	588
0.5	0.8	0.2	416875	590
0.8	0.2	0.5	326250	593
0.8	0.7	0.9	382500	596
0.7	0	0	482500	597
1	0.6	0.5	473125	585
0.5	0	1	426250	591
1	0.5	0.7	504375	590
0.5	0.2	0.3	448125	597
0.8	0.5	0.4	498125	593
0.6	0.3	0.8	510625	598
0.7	0.5	0.5	463750	589
0.9	0.4	0.4	345000	605
0.6	0.8	0.3	388750	602
0.7	0.4	0.6	379375	608
0.6	0.7	0.9	332500	601
0.8	0.2	0.4	338750	615
0.8	0.7	0	351250	600
1	0.4	0.8	370000	614
0.6	0.9	0.8	395000	606
0.6	0.2	0.1	438750	609
0.8	0.6	0.2	520000	608
0.8	0.1	0.8	479375	601
0.9	0.8	0.6	485625	613
0.9	0.3	0.1	441875	604
0.7	0.8	0.3	466875	612
0.7	0.1	0.7	432500	614
0.9	0.6	0.9	491875	607
0.5	0.5	0.5	420000	600
0.3	0.9	0.6	454375	611
0	0.3	0.9	435625	605
0.1	0.6	0	476250	606
0.4	0.1	0.3	429375	613
0.1	0.5	0.8	516875	612
0.5	0.1	0.8	423125	610
0.2	0.7	0.5	513750	608
0.2	0.2	0.1	457500	604
0.3	0.9	1	357500	603
0	0.3	0.5	366875	615
0.5	0.9	0	413750	609
0	0.4	0.3	335625	610

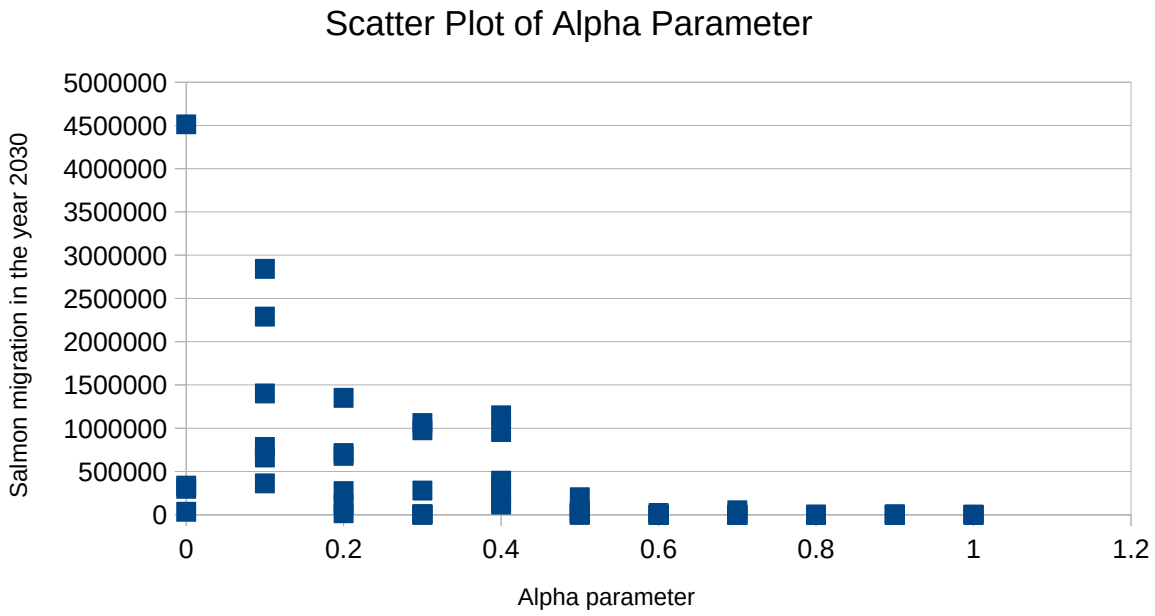
0.5	0.7	0.7	391875	603
0.2	0.4	0.6	341875	607
0.4	0.6	0.2	329375	602
0.3	0.4	0.5	376250	611
0.1	0.5	0.6	495000	595
0.4	0.1	0.7	451250	598
0.3	0.5	0.4	460625	592
0.4	0.2	0.1	507500	599
0.2	0.7	0.6	501250	585
0.2	0.2	1	488750	600
0	0.5	0.2	470000	586
0.4	0	0.2	445000	594
0.4	0.7	0.9	401250	591
0.3	0.3	0.8	320000	592
0.2	0.8	0.3	360625	599
0.1	0.1	0.4	354375	587
0.1	0.6	0.9	398125	596
0.3	0.1	0.7	373125	588
0.3	0.8	0.3	407500	586
0.1	0.3	0.1	348125	593



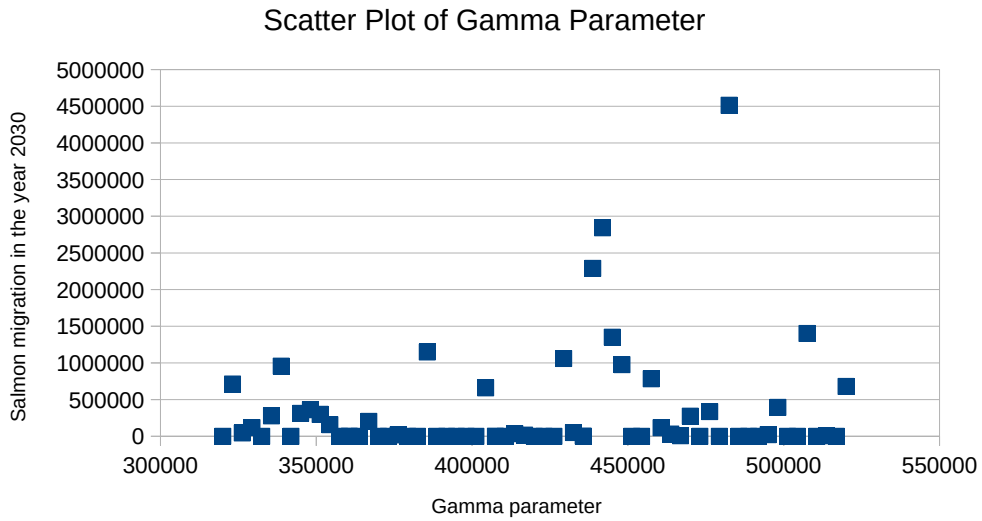
Appendix 5: *Scatter plot between the Pl parameter and salmon migration in the year 2030.*



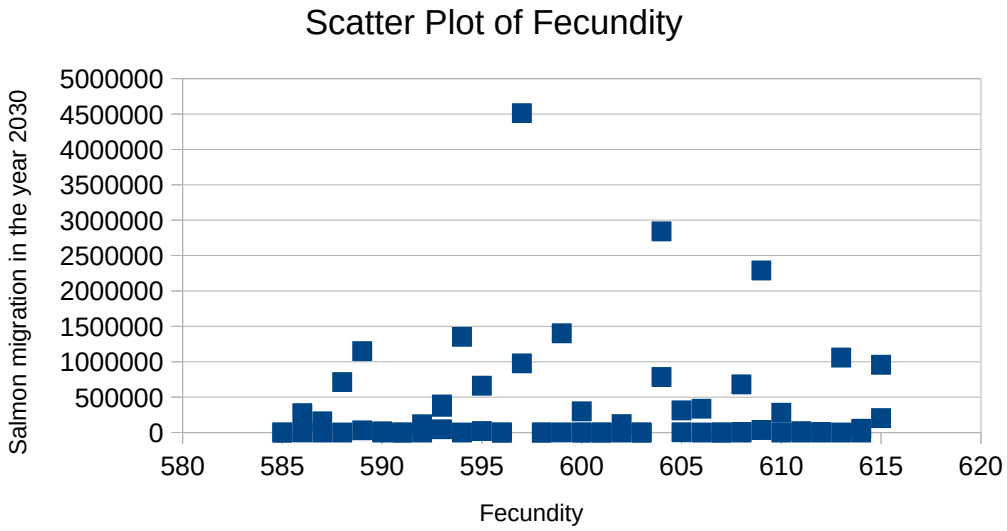
Appendix 6: *Scatter plot between the Pj parameter and salmon migration in the year 2030.*



Appendix 7: *Scatter plot between the alpha parameter and salmon migration in the year 2030.*



Appendix 8: *Scatter plot between the gamma parameter and salmon migration in the year 2030.*



Appendix 9: *Scatter plot between the fecundity parameter and salmon migration in the year 2030.*

Appendix 10: *The computational model made with the Rust programming language.*

```
use ndarray::Array2;
use ndarray::arr2;
fn main() {
  // AINEISTOT \\
  let measured_salmon_run: Vec<f64> = vec![
    31782.0, 17224.0, 23076.0, 59606.0, 53605.0, 101094.0, 58360.0, 98311.0, 40952.0, 47028.0,
    65520.0, 69149.0, 93121.0, 52030.0, 20245.0, 24627.0,
    45043.0, 0.0, 0.0, 0.0, 0.0, 0.0,
  ];
  let fishing_tornionjoki: Vec<f64> = vec![
    13400.0, 8600.0, 10100.0, 20000.0, 15200.0, 24200.0, 23200.0, 29200.0, 18000.0, 21100.0,
```

```
    20900.0, 23300.0, 30900.0, 19800.0, 10900.0, 10800.0, 10800.0, 10800.0, 10800.0, 10800.0,
    10800.0, 10800.0
  ];
  // COASTAL FISHING
  let fishing_coastal: Vec<f64> = vec![
    116333.0, 68618.0, 68867.0, 69057.0, 68335.0, 68589.0, 64771.0, 61873.0, 52386.0, 57208.0,
    57144.0, 52907.0, 52561.0, 38140.0, 31515.0, 28071.0,
  ];
  // OFFSHORE FISHING
  let fishing_offshore: Vec<f64> = vec![
    60464.0, 79088.0, 74400.0, 57522.0, 33420.0, 30875.0, 27976.0, 26576.0, 28271.0, 36249.0,
    34090.0, 23188.0, 34235.0, 10298.0, 5039.0, 4977.0,
  ];
  let mut fishing_sea: Vec<f64> =
    fishing_coastal.iter()
    .zip(fishing_offshore.iter())
    .map(|(a, b)| a + b)
    .collect();
  fishing_sea.append(&mut vec![33048.0, 33048.0, 33048.0, 33048.0, 33048.0, 33048.0]);
  let mut seal_amounts: Vec<f64> = vec![
    15313.0, 19627.0, 23941.0, 28255.0, 29496.0, 32244.0, 30285.0, 30070.0, 30348.0, 34235.0,
    38121.0, 40075.0, 41936.0, 36761.0, 45798.0, 48711.0, 48087.0,
  ];
  seal_amounts.append(&mut vec![49882.0, 51678.0, 53473.0, 55269.0, 57064.0]);
  let spwn = 600.0;
  let alpha = 0.6;
  let gamma = 420000.0;
  let p_j = 0.2;
  let p_l = 0.1;
  // MUUT ARVOT \\
  // Kutulohen selviytymistodennäköisyys
  let sps: f64 = 0.075;
  // Kudusta -> Smolt selviytymistodennäköisyys
  let fs = 0.01;
  // Hylkeiden tuottaman kuolleisuuden suhde meren ja joen välillä.
  let seal_weight_sea = 0.2;
  let seal_weight_river = 0.8;
  // LESLIE-MALLI \\
let leslie_matrix = arr2(&[
  // FROM: P1, P2, P3, P4, P5, M1, J1, M2, J2, M3, J3, M4+, J4+
  // TO: P1
  [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, spwn, 0.0, spwn, 0.0, spwn, 0.0, spwn],
  // TO: P2
```

```
    [1.0 - p_l, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    // TO: P3
    [0.0, 1.0 - p_l, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    // TO: P4
    [0.0, 0.0, 1.0 - p_l, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    // TO: P5
    [p_l, p_l, p_l, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    // TO: M1
    [0.0, 0.0, 0.0, 0.0, fs, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    // TO: J1
    [0.0, 0.0, 0.0, 0.0, 0.0, 1.0 - p_j, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    // TO: M2
    [0.0, 0.0, 0.0, 0.0, 0.0, p_j, sps, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
    // TO: J2
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0 - p_j, 0.0, 0.0, 0.0, 0.0, 0.0],
    // TO: M3
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, p_j, sps, 0.0, 0.0, 0.0, 0.0],
    // TO: J3
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0 - p_j, 0.0, 0.0, 0.0],
    // TO: M4+
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, p_j, sps, p_j, sps],
    // TO: J4+
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0 - p_j, 0.0],
]);
    let pop_length: usize = leslie_matrix.nrows();
    let sea_indices = vec![5, 7, 9, 11];
    let river_indices = vec![6, 8, 10, 12];
    let start_population: Array2<f64> = calc_start(1.0, 100, &leslie_matrix);
    let total_river_ratio: f64 = river_indices
        .iter()
        .map(|&i| start_population[[i, 0]])
        .sum::<f64>();
    let scale = measured_salmon_run[0] as f64 / (total_river_ratio);
    let start_population = &start_population * scale;
    let mut salmon_amounts = Vec::new(); // 0 = Female, 1 = Male
    for _ in 0..2 {
        salmon_amounts.push(start_population.clone() / 2.0);
    }
    let simulation_years: usize = measured_salmon_run.len();
    let mut calculated_salmon_run: Vec<f64> = vec![0.0; simulation_years];
    for i in 0..simulation_years {
        // Calculate sea fishing weight:
        let sea_amt: f64 = sea_indices.iter().map(|j| salmon_amounts[0][[*j, 0]] + salmon_amounts[1][[*j, 0]]).sum();
```

```
let fishing_sea_tornionjoki_weight: f64 = sea_amt / (sea_amt + gamma);
// Calculate fishing for year i:
let h_sea = fishing_sea[i] * fishing_sea_tornionjoki_weight;
let h_river = fishing_tornionjoki[i];
let total_sea_female = sea_indices.iter().map(|&k| salmon_amounts[0][[k, 0]]).sum::<f64>();
let total_sea_male = sea_indices.iter().map(|&k| salmon_amounts[1][[k, 0]]).sum::<f64>();
let total_river_female = river_indices.iter().map(|&k| salmon_amounts[0][[k, 0]]).sum::<f64>();
let total_river_male = river_indices.iter().map(|&k| salmon_amounts[1][[k, 0]]).sum::<f64>();
let h_sea_female = h_sea * total_sea_female / (total_sea_female + total_sea_male);
let h_sea_male = h_sea * total_sea_male / (total_sea_female + total_sea_male);
let h_river_female = h_river * total_river_female / (total_river_female + total_river_male);
let h_river_male = h_river * total_river_male / (total_river_female + total_river_male);
let mut fishing_female: Array2<f64> = Array2::zeros((pop_length, 1));
let mut fishing_male: Array2<f64> = Array2::zeros((pop_length, 1));
// Do not take first year fishing into account:
if i != 0 {
    distribute_value(h_sea_female,&sea_indices,&salmon_amounts[0],&mut fishing_female,);
    distribute_value(h_sea_male,&sea_indices,&salmon_amounts[1],&mut fishing_male,);
    distribute_value(h_river_female,&river_indices,&salmon_amounts[0],&mut fishing_female,);
    distribute_value(h_river_male,&river_indices,&salmon_amounts[1],&mut fishing_male,);
}
// Calculate predation by seals:
let s_sea = seal_amounts[i] * seal_weight_sea;
let s_river = seal_amounts[i] * seal_weight_river;
let seal_mortality_sea_female = s_sea * total_sea_female / (total_sea_female + total_sea_male);
let seal_mortality_sea_male = s_sea * total_sea_male / (total_sea_female + total_sea_male);
let seal_mortality_river_female = s_river * total_river_female / (total_river_female + total_river_male);
let seal_mortality_river_male = s_river * total_river_male / (total_river_female + total_river_male);
let mut seal_mortality_female: Array2<f64> = Array2::zeros((pop_length, 1));
let mut seal_mortality_male: Array2<f64> = Array2::zeros((pop_length, 1));
distribute_value(seal_mortality_sea_female,&sea_indices,&salmon_amounts[0],&mut seal_mortality_female,);
distribute_value(seal_mortality_sea_male,&sea_indices,&salmon_amounts[1],&mut seal_mortality_male,);
distribute_value(seal_mortality_river_female,&river_indices,&salmon_amounts[0],&mut seal_mortality_female,);
distribute_value(seal_mortality_river_male,&river_indices,&salmon_amounts[1], &mut seal_mortality_male,);
// Calculate next step of the Leslie model:
// At the same time split fecundity between male and female.
// FEMALE
let mut female_matrix = leslie_matrix.clone();
for j in &river_indices {
    female_matrix[[0, *j]] = spwn * 0.5;
}
salmon_amounts[0] = &salmon_amounts[0] - &fishing_female - alpha * &seal_mortality_female;
for j in &river_indices {
```

```
            if salmon_amounts[0][[*j, 0]] < 0.0 {
                salmon_amounts[0][[*j, 0]] = 0.0;
            }
        }
        for j in &sea_indices {
            if salmon_amounts[0][[*j, 0]] < 0.0 {
                salmon_amounts[0][[*j, 0]] = 0.0;
            }
        }
        salmon_amounts[0] = female_matrix.dot(&salmon_amounts[0]);
        // MALE
        let mut male_matrix = leslie_matrix.clone();
        for j in &river_indices {
            male_matrix[[0, *j]] = 0.0;
        }
        salmon_amounts[1] = &salmon_amounts[1] - &fishing_male - alpha * &seal_mortality_male;
        for j in &river_indices {
            if salmon_amounts[1][[*j, 0]] < 0.0 {
                salmon_amounts[1][[*j, 0]] = 0.0;
            }
        }
        for j in &sea_indices {
            if salmon_amounts[1][[*j, 0]] < 0.0 {
                salmon_amounts[1][[*j, 0]] = 0.0;
            }
        }
        salmon_amounts[1] = male_matrix.dot(&salmon_amounts[1]);
        salmon_amounts[1][[0, 0]] = salmon_amounts[0][[0, 0]];
        // Laske nousulohet vuodelle i:
        for sex in 0..2 {
            for j in &river_indices {
                calculated_salmon_run[i] += salmon_amounts[sex][[*j, 0]];
            }
        }
    }
    // Print results:
    println!("Calculated salmon runs (2009-{}): ", 2009 + calculated_salmon_run.len() - 1);
    for salmon_run in &calculated_salmon_run {
        println!("{} ", if salmon_run.floor() < 0.0 { 0.0 } else { salmon_run.floor() });
    }
}
// Mean Square Error
fn mse(measured: &Vec<f64>, calculated: &Vec<f64>) -> f64 {
```

```
    let mut sum: f64 = 0.0;
    let n: usize = measured.len();
    for i in 0..n {
        sum += (measured[i] - calculated[i]) * (measured[i] - calculated[i]);
    }
    return sum / (n as f64);
}
// Variance
fn var(nums: &Vec<f64>) -> f64 {
    let mut ans: f64 = 0.0;
    let n: usize = nums.len();
    let average: f64 = nums.iter().sum::<f64>() / n as f64;
    for i in 0..n {
        ans += (nums[i] - average) * (nums[i] - average);
    }
    return ans / (n as f64);
}
// Coefficient of Determination
fn r2(measured: &Vec<f64>, calculated: &Vec<f64>) -> f64 {
    return 1.0 - mse(measured, calculated) / var(measured);
}
// Calculate age distribution
fn calc_start(total_population: f64, years_to_run: usize, leslie: &Array2<f64>) -> Array2<f64> {
    let mut population: Array2<f64> = Array2::ones((leslie.nrows(), 1));
    for _ in 0..years_to_run {
        population = leslie.dot(&population);
    }
    let total: f64 = (0..leslie.nrows()).map(|k| population[[k, 0]]).sum::<f64>();
    let normalized: Vec<f64> = population
        .iter()
        .map(|&x| x * total_population as f64 / total)
        .collect();
    return Array2::from_shape_vec((leslie.nrows(), 1), normalized).unwrap();
}
// Split a scalar to vector taking into account which indices to split into.
fn distribute_value(input: f64, indices: &Vec<usize>, salmon_amount: &Array2<f64>, output: &mut Array2<f64>) {
    let total = indices.iter().map(|k| salmon_amount[[*k, 0]]).sum::<f64>();
    if total != 0.0 {
        for &k in indices {
            let h = input * salmon_amount[[k, 0]] / total;
            output[[k, 0]] = if h <= 0.0 { 0.0 } else { h };
        } } }
```